МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Кемеровский государственный университет» Институт Цифры

ДОМАШНЯЯ РАБОТА №5 ПО ДИСЦИПЛИНЕ

“Параллельные вычисления”

Задачи № 1, 2, 3, 4

студентов 3 курса

Сизов Максим Евгеньевич

Грачев Ярослав Евгеньевич

Направление 02.03.02 – Фундаментальная информатика и информационные технологии

Преподаватель:
к-т физ.-мат.наук, доцент
С.В. Стуколов
_____

Работа защищена:
“____”_____2026г.

с оценкой _____

Кемерово 2026

СОДЕРЖАНИЕ

1. Задача 1 3
2. Задача 2 5
3. Задача 3 6
4. Задача 4 8
Заключение 9
Литература 10

1. Задача 1

Постановка:

Напишите программу, используя блокирующие коммуникационные функции (MPI_Send, MPI_Recv), реализующую построчное распределение двумерного массива, расположенного в памяти 0-го процессора. Все остальные процессы выводят номер процесса и полученную строку от 0-го процесса. В момент запуска параллельного приложения соберите трассу выполнения программы и приведите рисунок трассы в отчете.

Реализация:

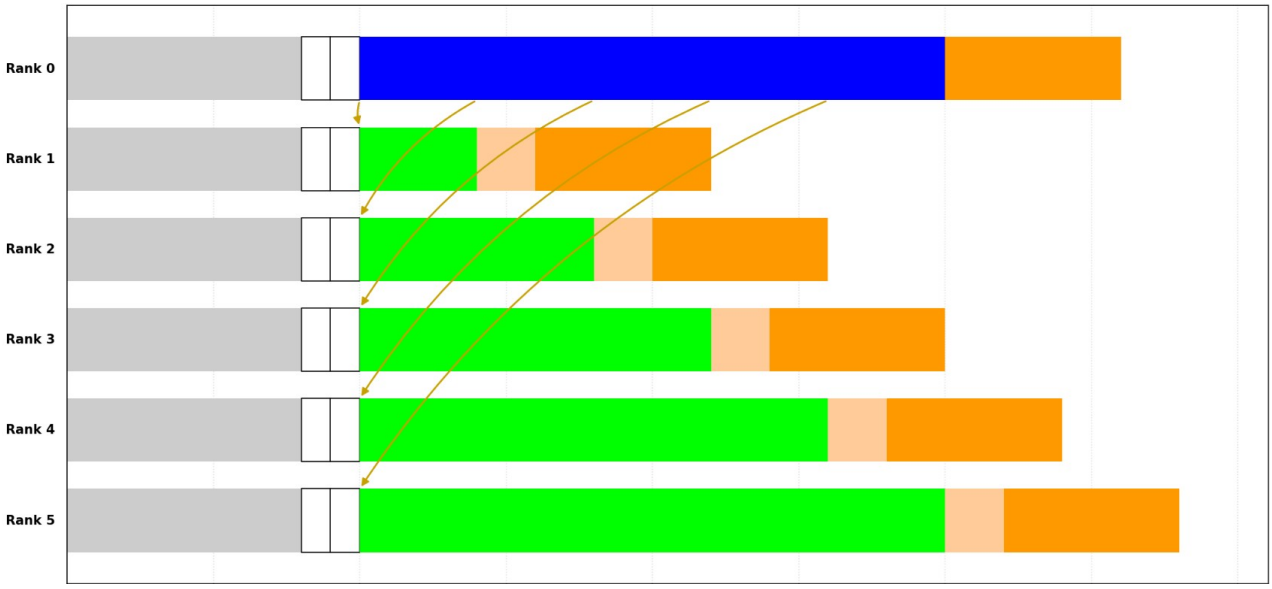
```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
	MPI_Init(&argc, &argv);
	int rank, size;
	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);
	MPI_Status stat;

	if (rank == 0) {
		int a[size][10];
		for (int i = 1; i < size; i++) {
			for (int j = 0; j < 10; j++) a[i - 1][j] = (i) * (j + 1);
			MPI_Send(a[i - 1], 10, MPI_INT, i, 777, MPI_COMM_WORLD);
			}
	}
	else {
		int b[10];
		MPI_Recv(b, 10, MPI_INT, 0, 777, MPI_COMM_WORLD, &stat);
		printf("rank: %d\n", rank);
		for (int i = 0; i < 10; i++) printf("%d ", b[i]);
		printf("\n");
	}

	MPI_Finalize();
	return 0;
}
```

```
PS C:\Users\makss\Documents\C++\nameless\bin\Debug> mpiexec -n 6 nameless.exe
rank: 3
3 6 9 12 15 18 21 24 27 30
rank: 4
4 8 12 16 20 24 28 32 36 40
rank: 1
1 2 3 4 5 6 7 8 9 10
rank: 5
5 10 15 20 25 30 35 40 45 50
rank: 2
2 4 6 8 10 12 14 16 18 20
```



2. Задача 2

Постановка:
Напишите программу, используя блокирующие коммуникационные функции (MPI_Send, MPI_Recv), реализующую построчную сборку двумерного массива: на каждом процессе задается одномерный массив, который передается на 0-й, 0-й принимает одномерный массив, сразу сохраняя в соответствующую строку двумерного массива.

Реализация:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
	MPI_Init(&argc, &argv);
	int rank, size;
	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);
	MPI_Status stat;

	if (rank != 0) {
		int a[10];
		for (int i = 1; i <= 10; i++) a[i - 1] = rank * i;
		MPI_Send(a, 10, MPI_INT, 0, 777, MPI_COMM_WORLD);
	}
	else {
		int b[size - 1][10];
		for (int i = 1; i < size; i++) MPI_Recv(b[i - 1], 10, MPI_INT, i, 777,
MPI_COMM_WORLD, &stat);
		for (int i = 0; i < size - 1; i++) {
			for (int j = 0; j < 10; j++) printf("%d ", b[i][j]);
			printf("\n");
		}
	}

	MPI_Finalize();
	return 0;
}
```

```
PS C:\Users\makss\Documents\C++\nameless\bin\Debug> mpiexec -n 4 nameless.exe
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
```

3. Задача 3

Постановка:
Напишите программу для распределения по три строки на каждый ненулевой процесс двумерного массива, расположенного в памяти 0-го процесса. На ненулевых процессах для приема объявите трехстрочный массив.

Реализация:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
	MPI_Init(&argc, &argv);
	int rank, size;
	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);
	MPI_Status stat;

	if (rank == 0) {
		int a[(size - 1) * 3][10];
		for (int i = 0; i < (size - 1) * 3; i++)
			for (int j = 0; j < 10; j++) a[i][j] = (i + 1) * (j + 1);
		for (int i = 1; i < size; i++) MPI_Send(&a[3 * (i - 1)][0], 30, MPI_INT,
i, 777, MPI_COMM_WORLD);
	}
	else {
		int b[3][10];
		MPI_Recv(b, 30, MPI_INT, 0, 777, MPI_COMM_WORLD, &stat);
		printf("rank: %d\n", rank);
		for (int i = 0; i < 3; i++) {
			for (int j = 0; j < 10; j++) printf("%d ", b[i][j]);
			printf("\n");
		}
	}

	MPI_Finalize();
	return 0;
}
```

```
PS C:\Users\makss\Documents\C++\nameless\bin\Debug> mpiexec -n 4 nameless.exe
rank: 1
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
rank: 2
4 8 12 16 20 24 28 32 36 40
5 10 15 20 25 30 35 40 45 50
6 12 18 24 30 36 42 48 54 60
rank: 3
7 14 21 28 35 42 49 56 63 70
8 16 24 32 40 48 56 64 72 80
9 18 27 36 45 54 63 72 81 90
```

4. Задача 4

Постановка:
Напишите программу для сборки по три строки с каждого процесса на 0- й, который осуществляет прием сразу в двумерный массив.

Реализация:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
	MPI_Init(&argc, &argv);
	int rank, size;
	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);
	MPI_Status stat;

	if (rank != 0) {
		int a[3][10];
		for (int i = 0; i < 3; i++)
			for (int j = 0; j < 10; j++) a[i][j] = (i + 1) * (j + 1) * rank;
		MPI_Send(a, 30, MPI_INT, 0, 777, MPI_COMM_WORLD);
	}
	else {
		int b[(size - 1) * 3][10];
		for (int i = 1; i < size; i++) {
			MPI_Recv(&b[3 * (i - 1)][0], 30, MPI_INT, i, 777,
MPI_COMM_WORLD, &stat);
		}
		for (int i = 0; i < (size - 1) * 3; i++){
			for (int j = 0; j < 10; j++) printf("%d ", b[i][j]);
			printf("\n");
		}
	}

	MPI_Finalize();
	return 0;
}
```

```
PS C:\Users\makss\Documents\C++\nameless\bin\Debug> mpiexec -n 4 nameless.exe
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
2 4 6 8 10 12 14 16 18 20
4 8 12 16 20 24 28 32 36 40
6 12 18 24 30 36 42 48 54 60
3 6 9 12 15 18 21 24 27 30
6 12 18 24 30 36 42 48 54 60
9 18 27 36 45 54 63 72 81 90
```

Заключение

В ходе выполнения данной лабораторной мы научились передавать двумерные массивы с помощью блокирующих функций Send-Recv.

Литература

1.Стуколов С.В. Параллельное программирование. Практикум